ICS211 : Design and Analysis of Algorithms

Course Faculty: Dr. Jisha Mariyam John

jishamj@iiitkottayam.ac.in

Comparing time efficiency

- We measure time efficiency only upto an order of magnitude
  - Ignore constants
- How do we compare functions with respect to orders of magnitude?

Asymptotic Analysis

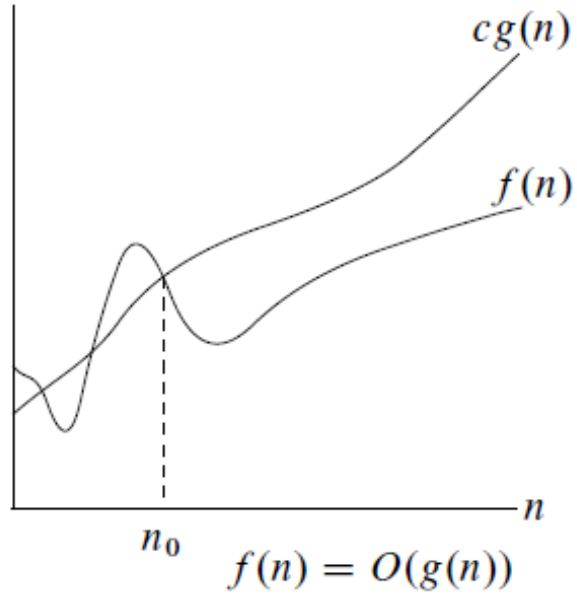
- We are concerned with How the running time of an algorithm increases, as the size of the input increases without bound.

Asymptotic notation

- The notations we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of natural numbers.

Upper bounds, “big O”

- f(n) = O(g(n)); there exist positive constants c and n_0 such that cg(n) is an upper bound for f(n) for n beyond n_0 .
- $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$

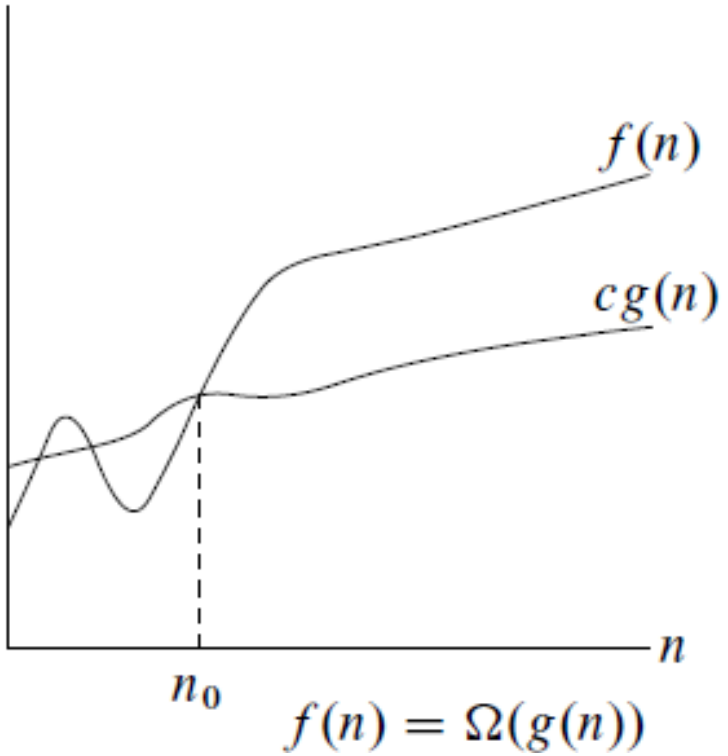


Examples:

- 100n+ 5 is O(n^2)
- A function f(n)=O(g(n)) if there exist constants c> and n_0 such that f(n)≤cg(n)for all n≥ n_0.
- Here,
  - f(n)=100n+5
  - g(n)=n2
- We need to find suitable values of c and n0.
- c=101, n_0 =5
- c=105, n_0 =1
- n0 and c are not unique!
- Of course, by the same argument, 100n+5 is also O(n)

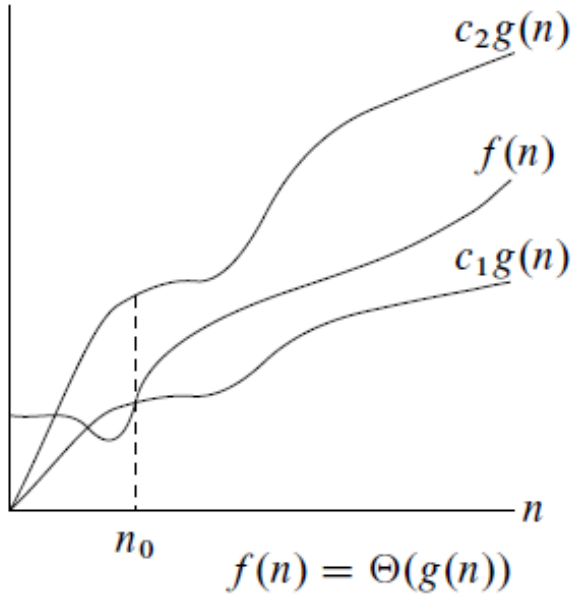
Lower bounds, Ω (omega)

- f(n) = Ω(g(n)) ;there exist positive constants c and n_0 such that cg(n) is an lower bound for f(n) for n beyond n_0 .
- 0 ≤ cg(n) ≤ f(n) for all n ≥ n_0



Tight bounds, Θ (theta)

- f(n) is Θ(g(n)) if it is both O(g(n)) and Ω(g(n))
- Find suitable constants c1, c2, and n_0 such that 0 ≤ c1g(n) ≤ t(n) ≤ c2g(n) for all n ≥ n_0



Examples

- Show that $2n^2 = O(n^3)$

$f(n) = 2n^2 \quad g(n^3) \qquad c, n_0$

$f(n) \leq c \cdot g(n) \quad \forall \; n \geq n_0$

$2n^2 \leq c \cdot n^3$

$2 \leq c \cdot n$

$n_0 = 1 \quad c = 2$

Examples

- Show that $2n^2 = O(n^3)$
  - f(n) = $2n^2$
  - g(n) = n^3
  - $2n^2 \leq c\, n^3$
  - $2 \leq c\, n$
  - With c=1 and n_0 =2 =>$2n^2 \leq c\, n^3$
  - So $2n^2 = O(n^3)$

f(n) = O(g(n)); there exist positive constants c and n_0 such that cg(n) is an upper bound for f(n) for n beyond n_0 .
$0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$

Examples

- $\sqrt{n}$ = Ω(lg n)

Examples

- $2^{n+1} = O(2^n)$

$f(n) = O(g(n))$ c, n_0 $f(n) \leq c \cdot g(n)$ $\forall n \geq n_0$

$f(n) = 2^{n+1}$

$g(n) = 2^n$

$2^{n+1} \leq c \cdot 2^n \quad \forall n \geq n_0$

$2^n \cdot 2^1 \leq c \cdot 2^n$

$\div$ by 2^n

$2 \leq c$

$n_0 = 1 \quad c = 2$

Examples

- $2^{2n} \neq O(2^n)$

$f(n) = 2^{2n}$ $g(n) = 2^n$

$f(n) = O(g(n))$ c, n_0

$f(n) \leq c \cdot g(n)$ $\forall n \geq n_0$

$2^n \leq c \cdot 2^n$ $\forall n \geq n_0$

$2^n \cdot 2^n \leq c \cdot 2^n$

$/\!/ \text{by } 2^n$

$2^n \leq c.$ $\Rightarrow$ There is no const with value always $> 2^n$

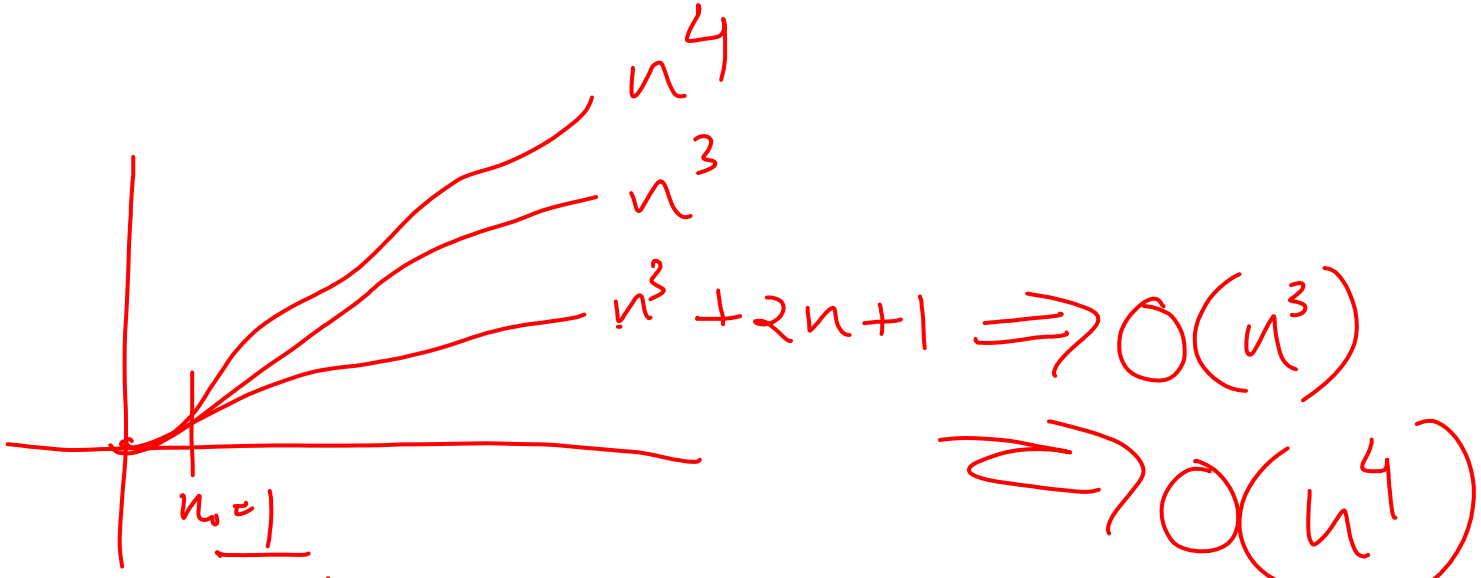
Examples

- f(n) = 3n+2 and g(n) = n. Is f(n) = O ($g(n)$)

Examples

- n^3 + 20 n + 1 = O (n^3)

$f(n) = O(n^3)$



Examples

- n^3 + 20 n + 1 = O (n^4)

Examples

- n^2+ n = O(n^3)

Examples

- Derive O notation for f(n) = n^2 + 2 n + 5

assume $g(n) = n^2$

Other notations

o-notation (little oh)

- We use o-notation to denote an upper bound that is not asymptotically tight.

$$o(g(n)) = \{f(n) : \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < cg(n) \text{ for all } n \geq n_0\}.$$

***ω*-notation (little omega)**

- We use ***ω*** -notation to denote a lower bound that is not asymptotically tight.

$$\omega(g(n)) = \{f(n) : \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq cg(n) < f(n) \text{ for all } n \geq n_0\}.$$

Properties

- Many of the relational properties of real numbers apply to asymptotic comparisons as well.

Transitivity:

$$f(n) = \Theta(g(n)) \text{ and } g(n) = \Theta(h(n)) \text{ imply } f(n) = \Theta(h(n)) ,$$
$$f(n) = O(g(n)) \text{ and } g(n) = O(h(n)) \text{ imply } f(n) = O(h(n)) ,$$
$$f(n) = \Omega(g(n)) \text{ and } g(n) = \Omega(h(n)) \text{ imply } f(n) = \Omega(h(n)) ,$$
$$f(n) = o(g(n)) \text{ and } g(n) = o(h(n)) \text{ imply } f(n) = o(h(n)) ,$$
$$f(n) = \omega(g(n)) \text{ and } g(n) = \omega(h(n)) \text{ imply } f(n) = \omega(h(n)) .$$

Reflexivity:

$$f(n) = \Theta(f(n)) ,$$
$$f(n) = O(f(n)) ,$$
$$f(n) = \Omega(f(n)) .$$

Symmetry:

$$f(n) = \Theta(g(n)) \text{ if and only if } g(n) = \Theta(f(n)) .$$

Transpose symmetry:

$$f(n) = O(g(n)) \text{ if and only if } g(n) = \Omega(f(n)) ,$$
$$f(n) = o(g(n)) \text{ if and only if } g(n) = \omega(f(n)) .$$